

This API design follows RESTful principles, focusing on security, idempotency for financial operations, and clear resource separation.

1. API Overview

- **Base URL:** `https://api.bankingsystem.com/v1`
- **Protocol:** HTTPS (TLS 1.3)
- **Data Format:** JSON
- **Versioning:** URI-based (`/v1/`)
- **Concurrency:** ETags used for optimistic locking on balance updates.

2. Authentication

- **Mechanism:** OAuth 2.0 + OpenID Connect (OIDC).
- **Flow:** Authorization Code Flow with PKCE.
- **MFA:** Headers require `X-MFA-Token` for sensitive actions (transfers > \$1,000, account settings).
- **JWT Claims:** `sub`, `role`, `exp`, `iat`, `jti` (for replay protection).

3. Endpoints

Account Management

- `GET /accounts` – List all accounts for the authenticated user.
- `GET /accounts/{id}` – Get specific account balance and details.

Transactions

- `POST /transactions` – Execute a fund transfer.
- `GET /transactions/{id}` – Retrieve status/history of a specific transaction.

Loan Management

- `POST /loans` – Submit a new loan application.
- `GET /loans` – List applications (Filter by status for Employees).
- `PATCH /loans/{id}` – Update status (Approve/Reject - Employee only).

AI & Insights

- `GET /insights/budget` – Retrieve AI-generated budgeting advice.

4. Request/Response Examples

POST /transactions (Transfer Funds)

Request Body:

```
{
  "from_account_id": "acc_123",
  "to_account_id": "acc_456",
  "amount": 500.00,
  "currency": "USD",
  "idempotency_key": "uuid-v4-string"
}
```

Response Body (201 Created):

```
{
  "transaction_id": "tx_987654321",
  "status": "COMPLETED",
  "timestamp": "2023-10-27T10:00:00Z"
}
```

5. Status Codes

| Code | Meaning | Usage |

| :--- | :--- | :--- |

| `200 OK` | Success | General read operations. |

| `201 Created` | Resource Created | Successful transaction or application. |

| `400 Bad Request` | Validation Error | Malformed input or logic violation. |

| `401 Unauthorized` | Missing/Invalid Token | User not logged in. |

| `403 Forbidden` | Insufficient Permissions | User lacks role to perform action. |

| `409 Conflict` | Idempotency violation | Duplicate transaction attempt. |

| `429 Too Many Requests` | Rate Limiting | Preventing brute force/DDoS. |

6. Validation & Error Handling

Validation Rules (Middleware)

- **Amount:** Must be `> 0`.
- **Currency:** Must follow ISO-4217 standard.
- **Idempotency:** The `idempotency\_key` is checked in Redis/Database to prevent double-charging within a 24-hour window.

Standard Error Response

```
{
  "error": {
    "code": "INSUFFICIENT_FUNDS",
    "message": "The account balance is insufficient for this transaction.",
    "request_id": "req-xyz-123",
    "details": {
      "current_balance": 150.00,
      "requested_amount": 500.00
    }
  }
}
```

7. Senior Architect Notes for Implementation

-
1. **Idempotency:** Every `POST /transactions` request **must** include an `Idempotency-Key` header. The server will cache the result of the request for 24 hours to return the original response if the client retries due to a timeout.
 2. **Audit Logging:** Every write-operation endpoint is wrapped in a decorator that writes to the `audit\_logs` table before the final transaction commit.
 3. **Performance:** All `GET` requests are paginated using `cursor`-based pagination to ensure consistent performance regardless of transaction volume.
 4. **Security Header:** All responses must include `Strict-Transport-Security`, `X-Content-Type-Options: nosniff`, and `Content-Security-Policy`.